

目录

大作业事项.....2

题目一：基于 TCP 通信的联邦学习系统.....3

题目二：基于 MCP 与 A2A 协议的分布式多智能体协同网络.....4

题目三：基于域名解析协议与机器学习的智能 DNS 拦截服务器6

题目四：基于深度学习的加密流量实时识别与监控系统.....8

题目五：基于 UDP 的应用层可靠传输与 AI 驱动拥塞控制协议实现.....9

题目六：基于 SDN 与机器学习的虚拟网络构建与智能流量调度12

题目七：融合 AI 智能助手的分布式即时聊天系统开发.....13

题目八：流式视频传输系统设计与 AI 驱动的智能码率自适应.....15

题目九：DDoS 攻击对抗与基于深度学习的智能防御系统设计16

题目十：基于经典协议的邮件客户端开发与机器学习反垃圾机制18

大作业事项

1. 分数占比

大作业分数占期末总评的 30%。大作业内分数分为基础分+附加分两部分，**基础分占比 75%，附加分占比 25%**。基础分为必做内容的完成度，附加分为所做扩展内容的分值总和，最多 25 分。扩展项可自由选择，鼓励多做。（选择“自定义开放性问题”的小组，须在期中提交问题 Proposal，阐述问题意义并预估分值。老师与助教将对其进行审核，并敲定最终分值。）

2. 开展形式

同学们自由组队，推荐为 **3~4** 人每组。本次大作业共 10 个题目，每个题目供至多 3 个组选择，选题链接：<https://www.kdocs.cn/l/cd1M0BcLi3vD>（选题时间：3 月 13 日 12:00-3.18 日 12:00. **切勿覆盖他人已填表格!!!**）。16 周课上 poster 和 demo 展示，现场打分。

3. 提交内容

主要分为两部分。

1) 项目海报。参照给出的学术会议 poster 形式，总结自己组的工作内容，制作一份海报。

2) 一份提交文件，包括项目报告文档，代码，以及可能有的演示材料，如截图、视频等，具体在每道题后面都有要求。

题目一：基于 TCP 通信的联邦学习系统

实验目标：

1. 掌握 TCP 网络编程原理及面向字节流的通信机制；
2. 解决 TCP 传输中的“粘包”问题，实现大容量模型数据的高效封装与拆解；
3. 理解联邦学习分布式架构与协同训练机制；
4. 实践中心聚合服务器与多边缘节点之间的网络状态监测与并发通信。

实验要求：

基础功能（必做）

1、模型与数据加载（直接使用提供的代码）：

使用提供的 `model.py`（轻量 CNN 模型定义）和分割好的数据集文件（如 `client1_data.pt`）。Client 节点严禁越权读取全局数据或其他节点的数据，仅限本地数据加载。

2、基于 TCP 通信传输模型参数：

使用 Python 原生的 `socket` 库建立 TCP 连接，严禁使用 HTTP 框架或高层 RPC 库。

由于神经网络参数（几百 KB）序列化为字节流后，超出了单次 `TCP recv(1024)` 的承受范围，直接接收会导致模型被截断或报错。须解决 TCP 面向字节流的“粘包”问题：

Client 端：在发送模型字节流之前，先发送一个固定长度（例如 8 字节）的数字，代表后续模型的总大小。

Server 端：开启多线程接收多个 Client 数据。对于单个 client，先接收 8 字节获取大小 L，然后利用 `while` 循环不断 `recv` 并拼接数据，直到精确收满 L 个字节，再进行反序列化。

3、联邦学习业务逻辑实现：

下发：Server 遍历已连接的 Client Socket，将全局模型广播发送。

训练：Client 接收完毕后，加载到本地 PyTorch 模型中，执行 1-2 个 Epoch 的训练，训练算法不限定。

聚合：Client 将训练后的模型权重发送回 Server；Server 需阻塞等待（或使用多线程/select），直到集齐 3 个 Client 的数据，将它们的权重进行加权计算，覆盖全局模型。

4、网络状态监测：

控制台实时输出轮次、Client_ID、广播耗时、等待/接收单节点更新耗时、Payload 大小等网络日志。系统最终需达到 85% 以上准确率。

扩展功能

去中心化通信（15 分）：废弃中心的 Server 节点，使 3 个 Client 组成环状网络。Client 1 训练完直接通过 TCP 发给 Client 2，Client 2 结合自己的数据训练后再发给 Client 3，探索去中心化网络拓扑下的运行模式。

复杂网络架构（15 分）：搭建云-边-端架构（1 主 Server、2 边缘 Server、6 Client），实现局部与全局两次聚合。

跨设备真实部署（10 分）：将一个或多个 Client 部署在局域网内的非 PC 边缘设备（如手机），通过真实 Wi-Fi 建立 TCP 连接进行协同训练。

处理网络延迟与掉队者（10 分）：在某个 Client 代码中人为加入 `time.sleep(5)` 模拟弱网。在 Server 端引入超时机制，如果超过一定时间未收到某个节点的数据，则主动放弃该节点，仅用已收到的节点进行模型聚合，保障系统进度。

学生自定义开放性问题（10-25 分）：提出一个与“计算机网络架构”或“通信效率”相关的扩展问题，并给出代码实现方案（如：网络传输前对模型进行 `gzip` 压缩或量化压缩以节省

带宽)。

技术栈要求:

该系统基础功能允许在单台或多台 PC 上通过“多进程”方式模拟(即在电脑上开 4 个终端,分别运行 1 个 Server 和 3 个 Client 的 Python 脚本),通过本地环回地址(127.0.0.1)建立 TCP 连接。严禁通过本地磁盘文件共享传递模型。

编程语言: Python 纯 Socket 编程。

核心库: socket(处理网络)、pickle / io.BytesIO(模型参数的序列化与反序列化)、torch, torchvision(模型计算)

分工建议:

- 客户端开发(加载模型数据与本地训练计算): 1 人
- 服务端开发(多线程通信、聚合算法与超时控制): 1 人
- 网络协议设计(TCP 收发封装、粘包处理): 1 人
- 测试与日志(网络状态监控、日志输出与指标统计): 1 人

实验提交内容:

1. 代码

Server 端和 Client 端的完整 Python 源码;
扩展功能实现代码。

2. 文档

设计文档: 系统架构图、TCP 封包结构定义;
测试报告: 网络监测日志截图, 最终模型准确率曲线;
用户手册: 多进程启动顺序及环境配置说明。

3. 演示

现场展示实时训练和推理全过程;
清晰展示控制台中输出的网络状态监测日志。

题目二: 基于 MCP 与 A2A 协议的分布式多智能体协同网络

实验目标:

1. 掌握分布式多智能体架构下 Agent-to-Agent (A2A) 的网络协同机制;
2. 深入理解并实践基于 HTTP/JSON-RPC 的 MCP (Model Context Protocol) 协议标准;
3. 实践进程间通信与端口映射管理, 实现网络流转与并发分发;
4. 探索基于 TCP 的自定义应用层协议设计。

实验要求:

基础功能(必做)

1、LLM 接入(使用提供代码):

使用提供的 llm_client.py, 里面封装好了免费的 LLM API, 只需调用 llm.chat("问题") 即可获取字符串回复。通过差异化的 Prompt 工程, 分别赋予主控与工作 Agent 相应的角色能力, 从而驱动多智能体系统的逻辑流转。

2、MCP 工具服务器构建(HTTP/JSON-RPC 协议实践):

理解 MCP 协议核心思想: Agent 不直接执行代码,而是通过标准网络协议请求 Server 执行。需要使用 Python 原生的 http.server 搭建 2 个独立的 MCP Server (分别监听不同端口,如 8001 和 8002)。

通信规范: 须使用 JSON-RPC 2.0 的格式规范。Agent 发送 POST 请求: {"jsonrpc": "2.0", "method": "get_weather", "params": {"city": "北京"}}, Server 收到后解析 JSON, 返回模拟数据: {"result": {"temp": "15° C", "condition": "晴"}}。

3、A2A 多智能体通信框架:

Agent 之间不能通过代码函数互相调用, 必须运行在不同的独立进程中, 通过网络互联。

在基础功能中, 规定 A2A 采用通用的 HTTP 协议 进行交互, 传递结构化的 JSON 报文。

比如

```
{
  "source": "coordinator",
  "target": "weather_agent",
  "task_id": "1001",
  "instruction": "查一下北京明天天气"
}
```

通信架构设计: 每个 Agent 节点既是“客户端”, 也是“服务端”:

主控 agent 监听主端口(如 9000), 接收用户请求, 向工作 Agent 端口发起 HTTP POST 任务分发, 并汇总最终结果。

工作 Agent 监听端口(如 9010), 暴露一个 /execute_task 的 HTTP 接口, 专门等待并接收主控 agent 派发的任务指令。收到任务被唤醒后, 首先主动向 MCP Server (如 8001) 发起 JSON-RPC 请求获取真实环境数据; 随后将数据交由本地 LLM 包装成自然语言回复后, 再次主动将最终结果 POST 传回主控 agent 的端口(如 9000)。

4、网络业务流转实现与记录:

实现完整通信链路: 用户提问 -> Coordinator -> (HTTP POST 任务分发) -> 天气 Agent -> (HTTP POST JSON-RPC) -> 天气 MCP Server -> (HTTP Response) -> 天气 Agent -> (HTTP 结果回传) -> Coordinator -> 汇总输出最终旅行方案。

系统的控制台须清晰地打印出每一次网络交互日志。要求能直观展示出 HTTP 请求的收发路径、端口号占用情况以及传递的 JSON Payload, 以此验证网络拓扑流转的正确性。

扩展功能

原生 TCP 替换 HTTP 实现 A2A 协议 (15 分): 基于原生 Python socket 设计一套基于 TCP 的应用层 A2A 通信协议。为了防止 TCP 面向字节流的“粘包”问题, 报文须包含自定义 Header 和 Body, 例如: Length (4 字节) + JSON Payload (包含 Sender, Receiver, Task_Content)。接收端必须精确使用 recv() 循环拆解数据包。

分布式网络容错 (10 分): 模拟某个 MCP Server 宕机或网络极度拥堵。要求 Agent 的网络请求必须设置超时机制。若调用失败, Agent 需能向 Coordinator 回复一个标准的错误报文(如 HTTP 500 概念), Coordinator 需及时处理该节点下线的情况, 防止系统卡死。

跨设备局域网部署 (15 分): 打破本地环回限制, 至少 2 个 Agent 运行在局域网内的不同同学的电脑/手机上, 通过真实内网 IP 互联完成协同。

完善旅行社新 Agent (15 分): 引入一个新 agent, 比如住宿 Agent、行程规划 Agent、餐饮 Agent、交通 Agent 等, 并为其配备独立的 MCP Server。新加入的 Agent 须与原有 Agent 产生网络执行依赖(串行或条件触发)。例如: Coordinator 必须先等待“票务 Agent”通过网络传回到达时间, 才能通过网络唤醒“酒店 Agent”去执行预订。控制台需清晰打印出网络

交互信息。

Agent 注册中心（一个 15，两个 25 分）：由原来的网络拓扑加入一个或两个 Agent 注册中心。每个 Agent 启动时，把自己的能力描述、网络地址注册到注册中心；主 Agent 通过查询注册中心来寻找合适的 Agent 并完成后续的交互逻辑。

学生自定义开放性问题（10-25 分）：提出一个与“计算机网络架构”或“通信效率”相关的扩展问题，并给出代码实现方案。（比如让网络拓扑更复杂）

技术栈要求：

该系统允许在单台/多台 PC 上通过开启 5 个终端进程（Coordinator, Agent1, Agent2, MCP-Server1, MCP-Server2）模拟。

编程语言：Python。

网络核心库：socket/struct（实现 A2A TCP 通信与拆包）、http.server（MCP HTTP 服务器）

AI 模块：课程提供基于统一 API 的极简 LLM 封装模块。

分工建议：

- 系统主控端开发（Coordinator 逻辑与任务分发引擎）： 1 人
- 业务 Agent 开发（工作 Agent 封装与 Prompt 调优）： 1 人
- MCP 工具服开发（HTTP/JSON-RPC 接口处理与模拟数据）： 1 人
- 底层通信与测试（TCP/HTTP 协议封装及全局日志管控）： 1 人

实验提交内容：

1. 代码

主控、业务 Agent 及 MCP Server 进程的全部源码；

扩展功能代码。

2. 文档

设计文档：系统进程通信网络拓扑图、A2A JSON 格式字典；

测试报告：不同角色的 Prompt 设计效果，网络容错处理结果；

操作手册：多进程启动脚本及使用说明。

3. 演示

现场演示用户完整 Prompt 交互至输出旅行方案的过程；

控制台实时滚动展示进程间的每一次 HTTP/TCP 收发日志。

题目三：基于域名解析协议与机器学习的智能 DNS 拦截服务器

实验目标：

1. 掌握 DNS 协议报文格式及核心记录类型（A/AAAA/CNAME/MX）；
2. 实现递归查询与迭代查询的域名解析流程及本地缓存机制；
3. 探索机器学习算法在网络安全（如恶意域名检测）中的应用；
4. 实践 DNS 劫持/拦截机制与智能预加载调度策略。

实验要求：

基础功能（必做）

1、DNS 报文解析与响应：

解析 DNS 请求报文（UDP 53 端口），提取查询域名与记录类型；

记录支持：至少支持 A 记录（IPv4）、AAAA 记录（IPv6）、CNAME 记录（别名）。

2、递归查询与缓存：

向公共 DNS（如 8.8.8.8）或根域名服务器发起迭代/递归查询，最终获取目标记录；

缓存机制：缓存查询结果并遵循 TTL 过期时间（如缓存命中时直接构造响应包返回）。

3、客户端查询工具：

实现类似 dig 的 DNS 查询工具：支持指定自定义的 DNS 服务器发送查询请求（如 dig @127.0.0.1 example.com A）；

解析并输出 DNS 响应报文的详细内容（包括 Answer/Authority/Additional 部分）。

扩展功能

智能恶意域名拦截（25 分）：使用网络上公开的数据集（白样本推荐 Tranco Top 1M，黑样本搜索“DGA dataset”关键词，如 ExtraHop DGA Detection Training Dataset、360 Netlab DGA Dataset 等，使用子集减少训练时间），集成轻量级机器学习分类器（如随机森林、SVM 或 MLP）。通过提取域名的字符串特征（如元音字母比例、信息熵、n-gram 特征等），判断请求域名是否为 DGA（域名生成算法）恶意僵尸网络域名。若是，则 DNS 服务器直接拒绝解析或返回 Sinkhole 地址（如 0.0.0.0），实现网络安全防护；

基于统计与关联规则的启发式预加载（15 分）：打破被动的 TTL 缓存机制，维护一个滑动时间窗口，实现简单的“伴随解析”发现机制（例如：统计发现当局域网主机请求 www.taobao.com 时，往往在 1 秒内紧接着请求 g.alicdn.com）。当再次命中前置域名时，DNS 服务器在后台主动提前发起后续域名的解析并放入本地缓存，从而降低用户的多域名加载延迟；

DNSSEC 签名验证（15 分）：验证 RRSIG 签名链，确保域名解析的完整性与真实性（需集成相关加密算法）；

负载均衡配置（10 分）：通过 DNS 轮询返回多个 A 记录，实现基础的网络负载均衡（如同一域名每次请求返回不同的 IP 排列顺序）；

学生自定义开放性问题（10-25 分）：自己提出一个与“计算机网络架构”或“通信效率”相关的扩展问题，并给出代码实现方案。

技术栈要求：

编程语言：Python（推荐使用 dnslib 库解析，或纯 socket 手动解析二进制报文）；

机器学习：Scikit-learn 或 PyTorch（用于恶意域名特征提取与分类器训练）；

网络工具：Wireshark（抓包分析）、dig/nslookup（测试工具）；

数据存储：SQLite（缓存记录）、JSON（本地域名配置）。

分工建议：

- 协议开发（实现 DNS 报文解析、递归查询与基础缓存）： 1-2 人
- AI 拦截开发（提取特征、训练恶意域名检测模型并集成至服务器）： 1 人
- 客户端与高级功能（构建查询工具、分析日志或预加载设计）： 1 人
- 测试与展示（抓包分析、拦截率与缓存命中率统计）： 1 人

实验提交内容：

1. 代码

DNS 服务器源码（含缓存与查询逻辑）与 客户端查询工具源码；
机器学习特征提取代码、模型训练脚本及预训练模型权重。

2. 文档

设计文档：DNS 报文结构图、递归查询流程图、恶意域名分类模型的特征工程说明；
测试报告：查询延迟对比（缓存命中 vs 未命中）、模型拦截准确率及误报率评估；
操作手册：服务器配置说明及环境依赖要求。

3. 演示

展示域名递归解析与本地缓存生效的全过程（结合 Wireshark 抓包）；
现场请求一个典型的 DGA 恶意域名（如 eqwzjxk.com），展示 DNS 服务器利用 AI 识别并将其解析拦截至 0.0.0.0 的全过程。

题目四：基于深度学习的加密流量实时识别与监控系统

实验目标：

1. 深入理解传输层 TCP 连接机制及代理服务器（Proxy）的工作原理；
2. 掌握加密流量（HTTPS/TLS）在无法解密负载时的特征提取与序列化方法；
3. 构建 1D-CNN（一维卷积神经网络）模型进行端到端的网络流分类；
4. 开发实时的流量监控组件，验证深度学习模型在真实复杂网络环境中的泛化能力。

实验要求：

基础功能（必做）

1、TCP/HTTP 代理服务器：

使用 Python 原生 socket 编写一个 TCP 代理服务器，能够接收本机客户端（如浏览器）的请求，将其转发至目标公网服务器，并将服务器响应回传给客户端，保障正常的网络连通性。

2、流量特征提取与预处理：

代理服务器在转发流量时，实时截取 TCP 连接的“前 N 个数据包的数据包长度（Packet Length）”，生成固定长度的序列（如 N=100，不足补 0，超过截断）。将长度数值归一化到 [0, 1] 区间，作为神经网络的输入。

3、离线模型训练：

下载公开的加密流量数据集（如 ISCX VPN-nonVPN 或 CIC-IDS2017 的子集，仅需区分 3~4 类常见应用如 Video、Chat、File Transfer）。编写脚本按五元组（源 IP、目的 IP、源端口、目的端口、传输层协议）对 PCAP 进行流切分，提取前 N 个包长度特征进行清洗与归一化。使用 PyTorch 构建一个包含卷积层（Conv1D）和全连接层的分类模型进行训练，输出模型准确率。

扩展功能

在线实时推理（25 分）：将训练好的 CNN 模型无缝集成到 TCP 代理服务器代码中。当新的 TCP 代理连接建立并积累够前 N 个数据包后，实时将特征送入模型进行在线推理，并在代理服务器控制台实时打印出当前连接的应用类型预测结果（如 "Stream Type: Video"）；
实时流量监控看板（25 分）：结合代理服务器的实时截获数据，将抓取的流量实时输入模型，并在 Web 界面（如使用 Streamlit 开发前端）动态展示当前局域网内的应用分布（如：

实时动态饼图显示当前 60% 流量为视频，30% 为聊天）；

模型可视化与对比实验(10 分)：绘制训练过程的 Loss/Accuracy 曲线图及混淆矩阵热力图；可引入一个简单的 MLP 或 LSTM 模型对比分类效果和推理延迟；

对抗网络抖动测试(10 分)：在代理转发链路中人为加入随机延迟或包大小扰动(Padding)，观察并评估 AI 模型识别准确率的抗干扰能力；

学生自定义开放性问题(10-25 分)：自己提出一个与“计算机网络架构”或“通信效率”相关的扩展问题，并给出代码实现方案。

技术栈要求：

网络核心库：Python socket, select 或 threading（用于并发代理转发的阻塞处理）；

深度学习框架：PyTorch（推荐）或 TensorFlow/Keras；

数据处理：Pandas, Numpy, Scikit-learn；

前端：Streamlit 或 Flask/Echarts

测试工具：Wireshark（抓包验证代理转发机制与连通性）。

分工建议：

- 代理底层开发（实现高并发的 TCP 代理转发机制）： 1-2 人
- 模型训练开发（清洗公开数据集、搭建并训练 CNN 模型）： 1-2 人
- **系统集成与前端**（实现实时推理逻辑、Web 可视化看板集成）： 1 人
- 测试与文档（撰写实验报告、绘制图表及对抗测试评估）： 1 人

实验提交内容：

1. 代码

代理服务器源码（Proxy.py）；

离线数据预处理脚本与深度学习模型训练源码（含注释）；

预训练好的模型权重文件（.pth 或 .h5）。

2. 文档

设计文档：代理服务器并发架构图、数据预处理流程、1D-CNN 模型结构参数；

测试报告：训练集/测试集准确率曲线、混淆矩阵图分析；

操作手册：如何配置浏览器或系统代理连接至该程序，以及实时监控看板的启动方法。

3. 演示

展示代理服务器的正常网络转发功能（浏览器配置代理后能正常上网）；

运行实时监控看板，小组成员现场打开浏览器访问特定的视频网站或聊天软件，展示系统在数秒内自动更新应用类别标签与流量占比饼图的过程

题目五：基于 UDP 的应用层可靠传输与 AI 驱动拥塞控制协议实现

背景介绍：

真实的 TCP 拥塞控制算法固化在操作系统的内核栈中，修改门槛极高。随着互联网的发展，谷歌等公司提出了基于 UDP 的 QUIC 协议，将可靠传输和拥塞控制移到了应用层。

本实验要求学生模拟 QUIC 的思想，基于纯 UDP Socket，在 Python 应用层从零实现一个带有“确认重传（ACK）”和“拥塞窗口（CWND）”机制的可靠传输协议。

学生将废弃传统的 AIMD（加性增乘性减）规则，引入强化学习 Q-Learning 算法，在线训练一个智能发送端。发送端通过实时监测应用层的 RTT 和丢包情况，利用 Q-Table 动态调整 UDP 的发送速率（CWND），最终在自己编写的协议上实现高吞吐、低延迟的传输。

实验目标：

1. 深入理解传输层协议内核，掌握基于原生 UDP 在应用层封装可靠传输机制；
2. 掌握窗口机制（CWND）、滑动队列与重传超时（RTO）的构建方法；
3. 实践网络状况的数字化表征（RTT 平滑采集、丢包率统计）；
4. 综合应用经典 AIMD 拥塞控制基线与前沿强化学习（如 Q-Learning）动态控制策略。

实验要求：

基础功能（必做）

1、应用层可靠传输协议建立：

自定义封包：设计 UDP 报文应用层，Sequence Number (4 字节) + Timestamp (8 字节) + Payload (1024 字节)。

确认与超时重传：Receiver 收到包后立刻回复携带 ACK Number 的 UDP 报文。Sender 需维护一个“未确认队列”，开启定时器，若超过设定的 RTO（重传超时时间）未收到 ACK，则判定为丢包并触发重传。

RTT 采样：Sender 收到 ACK 后，利用 当前时间 - 报文中的 Timestamp 计算出该数据包的 RTT，并平滑计算 SRTT。

2、虚拟瓶颈链路封装：

在 Sender 的 sendto() 外层构建模拟器。

模拟器设定一个固定带宽（如 100 包/秒，即每 10ms 漏出一个包发往真实网卡）和一个有限队列（如最大缓存 20 个包）。

当应用层瞬间把 CWND 大小的数据（例如 30 个包）塞给模拟器时：

产生真实延迟（排队）：前 20 个包进入虚拟队列。第 1 个包立刻发出，第 20 个包需要排队等待 $20 \times 10\text{ms} = 200\text{ms}$ 才会被真实 sendto() 发出。这模拟了拥塞导致的 RTT 飙升。产生拥塞丢包（溢出）：后面的 10 个包因为虚拟队列已满（超出 20 的容量），被模拟器直接抛弃，不执行 sendto。这模拟了缓存溢出。

3、实现传统 AIMD 拥塞控制基线：

实现类似 TCP Reno 的规则作为对照组：

初始化 CWND = 1（允许同时发送 1 个包，即未确认包数量上限）。

每收到一个 ACK：CWND += 1 / CWND（拥塞避免：加性增）。

一旦发生超时丢包：CWND = max(1, CWND / 2)（拥塞惩罚：乘性减）。

记录该策略下的网络指标。

4、Q-Learning 智能拥塞控制器设计：

抛弃上述 AIMD 规则，利用 Q-Table 动态控制 CWND：

状态空间（State）：统计过去 1 个 RTT 内的网络特征，将其离散化为 6 个状态：RTT 趋势（变大/变小/平稳）× 丢包事件（发生/未发生）。

动作空间（Action）：每个 RTT 周期结束时，AI 决定下一个周期的行为：0 (CWND 保持), 1 (CWND + 1), 2 (CWND / 2)。

奖励函数 (Reward): 设计一个复合奖励公式, 例如: $R = \alpha \times \text{本轮成功吞吐量} - \beta \times \text{平均}$

$\text{RTT} - \gamma \times \text{丢包数量}$ 。让 AI 学会既要发得多, 又不能让网络排队变卡。

在线学习: 利用 ϵ -greedy 策略探索网络, 并使用 Bellman 公式实时更新 Q-Table 矩阵。AI 可能无法在单次传输中收敛, 要求灵活调整 ϵ 值, 设计多轮循环训练机制。

5、数据记录与可视化:

程序运行结束后, 使用 matplotlib 绘制对比图表:

CWND 随时间的变化曲线图 (AIMD 的“锯齿波”与 Q-Learning 收敛后的“平滑波”)。
吞吐量与延迟柱状图 (对比两种算法的平均 Mbps 和 平均 RTT)。

扩展功能

动态网络突变响应 (10 分): 在传输到中途时, 人为在代码中将 VirtualLink 的带宽突然减半。观察 AIMD 恢复耗时, 及 Q-Learning 是否能遗忘旧策略快速收敛。

快速重传与乱序处理 (15 分): 基础功能的 UDP 重传仅依赖超时, 耗时极长。要求在应用层实现类似经典 TCP 的“连续收到 3 个重复 ACK 立即触发重传”机制, 并正确处理乱序到达的数据包, 提升网络利用率。

深度强化学习 (引入 DQN/PPO 控制连续状态) (25 分): 废弃离散的 Q-Table, 将精确的连续浮点数 (当前精确 RTT、丢包率百分比、当前 CWND 大小) 作为输入, 利用 PyTorch 构建一个简单的神经网络模型, 直接输出动作甚至连续的 CWND 乘数系数。探索神经网络在复杂网络状态下的控制潜力。

学生自定义开放性问题 (10-25 分): 自己提出一个与“计算机网络架构”或“通信效率”相关的扩展问题, 并给出代码实现方案。

技术栈要求:

建议在单台电脑上打开两个独立的终端窗口, 分别作为独立进程运行 Sender.py 和 Receiver.py。双方基于本地环回地址 (127.0.0.1) 和不同端口进行 Socket 通信。

编程语言: Python 3。

网络核心库: socket (原生 UDP), threading (发送与接收并发处理), time (高精度时间戳与超时控制)。

数据计算: numpy (用于 Q-Table 矩阵运算与状态空间更新)。

可视化库: matplotlib (绘制性能对比图)。

分工建议:

- 可靠传输底层架构 (Header 封装、定时重传、多线程控制): 1 人
- 虚拟网络拓扑模拟 (漏斗模型与队列溢出模拟): 1 人
- 智能拥塞算法核心 (AIMD 及 Q-Learning 状态/奖励函数设计): 1 人
- 测试与可视化 (Matplotlib 动态制图与数据对比): 1 人

实验提交内容:

1. 代码

Sender.py 与 Receiver.py 及封装的虚拟链路类源码;

预训练的 Q-Table/模型权重文件；

2. 文档

设计文档：应用层 UDP 协议头图解、Q-Learning 奖励函数数学模型；

测试报告：Q-Learning 收敛曲线，AIMD 与 AI 算法吞吐量详细对比；

操作手册：代码执行顺序、环境包安装要求。

3. 演示

现场展示确认重传和拥塞控制功能的有效性；

实时展示并对比预训练的 Q-Learning 策略与 AIMD 策略下 CWND 演变的图表曲线。

题目六：基于 SDN 与机器学习的虚拟网络构建与智能流量调度

实验目标：

1. 掌握 SDN 架构的核心原理及 OpenFlow 协议的基本操作；
2. 实践使用 Mininet 搭建网络拓扑并连接控制器（如 Ryu 或 OpenDaylight）；
3. 学习通过编程调用控制器 REST API 实现流表下发与管理；
4. 理解 VLAN 划分与网络隔离的实现机制；
5. 初步掌握基于网络状态数据的流量分析与预测方法。

实验要求：

基础功能（必做）

1、拓扑与控制器配置：

使用 Mininet 创建包含至少 2 台交换机（支持 OpenFlow 1.3 协议）和 4 台主机的拓扑，并保存为 Python 脚本；

连接 Ryu 或 OpenDaylight 控制器，确保控制器能够识别拓扑中的交换机；

通过 `ovs-ofctl` 命令手动添加流表，实现以下通信规则：

h1 与 h3 互通，h2 与 h4 互通；

其余主机间禁止通信；

利用控制器下发流表，为 h1-h3 分配 VLAN 10，h2-h4 分配 VLAN 20，并在交换机端口上实现 VLAN 标签的打标签（Push）和剥离（Pop）操作。

2、流量监控与预测：

编写 Python 脚本，调用 Mininet 主机节点的网络测试工具（如 `iperf` 或 `D-ITG`），在主机间（如 h1 到 h3）自动生成随时间动态波动的背景流量（例如：通过脚本控制 `iperf` 速率，模拟出类似正弦波或早晚高峰的突发流量），为预测模型提供有意义的数据源；

编写控制器应用程序（Python），通过 OpenFlow 协议定期轮询交换机端口统计信息（Port Stats）；

实现一个基础的流量预测模块：采集链路历史流量数据，使用统计学算法（如滑动平均 Moving Average 或简单线性回归），实时计算并输出下一时刻的链路负载预测值。

3、流表验证与测试：

使用 `pingall` 命令测试主机连通性，并通过 Wireshark 抓包验证 VLAN 标签是否符合要求；

查看交换机流表并截图记录（如 `ovs-ofctl dump-flows s1`）；

在控制台展示流量监控数据的实时日志及预测结果。

扩展功能

智能路由调度（25 分）：基于基础功能中的流量预测结果，实现闭环控制。当预测到某条链路即将拥塞时，控制器自动计算新路径并下发流表（Flow Mod），将流量无缝牵引至空闲链路，实现动态负载均衡；

动态网络控制（15 分）：调用控制器 API 实现流表硬超时（如中断 h1-h3 通信 20 秒）；

网络性能限制（10 分）：在拓扑中为链路设置带宽、延迟、丢包率等参数（如 h1 与 s1 间带宽 10Mbps、延迟 5ms）；

胖树拓扑构建（25 分）：编写 Python 代码生成数据中心胖树拓扑，并验证多路径负载均衡。

学生自定义开放性问题（10-25 分）：提出一个与“计算机网络架构”或“通信效率”相关的扩展问题，并给出代码实现方案。

技术栈要求：

工具：Mininet、Ryu/OpenDaylight 控制器、Wireshark、iperf；

编程语言：Python（用于拓扑生成、控制器 API 调用及预测算法实现）；

通信协议：OpenFlow；

数据存储：JSON 文件（用于流表配置）或 CSV（用于记录流量日志）。

分工建议：

- 拓扑搭建（编写 Mininet 脚本并配置链路参数） 1-2 人
- 控制器开发（实现 Ryu/OpenDaylight 的 API 调用及流量预测模块） 1-2 人
- 流表管理（设计流表规则并验证通信隔离） 1 人
- 测试与优化（验证预测准确性及路由切换效果） 1 人

实验提交内容：

1. 代码

Mininet 拓扑脚本（含性能限制代码）；

控制器 API 调用程序（含流量监控与预测算法脚本）。

2. 文档

设计文档：系统架构图、流表规则定义、流量预测算法原理说明；

测试报告：连通性测试截图、Wireshark 抓包分析、流量预测数据对比图；

操作手册：环境配置步骤与脚本运行说明。

3. 演示

实时展示拓扑生成与流表下发过程；

展示控制台输出的实时流量预测数值，以及（若选做）拥塞发生时的自动路由切换效果。

题目七：融合 AI 智能助手的分布式即时聊天系统开发

实验目标：

1. 掌握基于 TCP/UDP 的网络通信原理；
2. 实践多线程编程与并发控制；
3. 理解客户端-服务器架构设计；
4. 实现基础网络协议设计（如心跳包、文件传输协议）；
5. 探索在网络应用中集成第三方 AI 服务（如大模型 API）的方法。

实验要求：

基础功能（必做）

1、客户端功能：

用户注册/登录（用户名唯一性校验）；
一对一文字聊天（支持私聊）；
群组聊天（支持创建/加入/退出群组）；
用户在线状态显示（在线/离线）；
消息历史记录（本地存储或服务器存储）。

2、服务器功能：

用户管理（注册信息存储、在线状态同步）；
消息路由（将消息转发给指定用户/群组）；
支持至少 50 个客户端并发连接；
心跳机制检测客户端离线。

扩展功能

LLM 智能助手接入（25 分）：

在服务端集成大语言模型 API（如通义千问、智谱清言等免费接口），当用户在群聊中发送特定指令（如 @AI）时，服务端调用模型接口获取回答并转发回群组；

智能内容审核（15 分）： 利用轻量级 NLP 情感分析库或提示词工程，自动识别聊天室内的违规、恶意攻击性言论，并进行屏蔽或警告；

文件传输（15 分）： 支持断点续传的文件发送与接收功能；

消息撤回功能（10 分）： 实现服务器端协调的消息撤回逻辑（如 2 分钟内可撤回）；

跨平台兼容性（15 分）： 支持 Windows/Linux/macOS 客户端（或 命令行 CLI 与 GUI 互通）；

学生自定义开放性问题（10-25 分）： 自己提出一个与“计算机网络架构”或“通信效率”相关的扩展问题，并给出代码实现方案。

技术栈要求：

编程语言：Python/Java/C++（推荐 Python）；

通信协议：TCP 用于文字/文件传输，UDP 可选用于状态广播，HTTP 用于调用 AI API；

多线程：必须使用多线程/异步 IO 处理并发请求；

数据存储：SQLite/JSON 文件或内存数据库（如 Redis）。

分工建议：

- 服务器开发（处理连接、路由、用户管理及 AI API 调用） 1-2 人
- 客户端开发（实现 UI、消息发送/接收逻辑） 1-2 人

- 协议设计（定义通信协议格式与接口） 1 人
- 测试（压力测试、性能调优及 AI 响应测试） 1 人

实验提交内容：

1. 代码

客户端和服务端完整源码；

包含清晰的代码注释和模块说明。

2. 文档

设计文档：系统架构图、协议定义（消息格式示例）、AI 功能实现逻辑；

测试报告：并发压力测试结果（如 100 用户同时在线）、智能回复/审核功能测试截图（若选做）；

用户手册：客户端安装与使用说明。

3. 演示

现场展示 3 个客户端同时通信（文字+文件传输）；

展示 @AI 智能回复效果或违规言论自动过滤效果（若选做）；

模拟服务器断线重连场景。

题目八：流式视频传输系统设计与 AI 驱动的智能码率自适应

实验目标：

1. 掌握基于 TCP 的流式视频传输原理与分片封装技术；
2. 实现客户端与服务端的动态交互及视频实时播放功能；
3. 设计基础的码率自适应算法（ABR）以应对网络波动；
4. 探索流式视频的高效路径优化与数据分片重组机制。
5. 探索人工智能在流媒体网络拥塞控制中的优化应用。

实验要求：

基础功能（必做）

1、服务端功能：

支持视频分片处理：将长视频分割为固定大小的片段（如每段 5 秒），封装为 TS 格式文件；

基于 TCP 协议实现分片传输，支持并发处理多个客户端请求；

记录每个分片的发送时间与码率，并生成传输日志。

2、客户端功能：

动态请求视频分片：循环向服务端请求分片文件（如 ocean-1080p-8000k-0.ts），支持分片序号拼接；

接收分片后调用播放器库实现实时播放；

基础码率自适应：实现一个简单的基于规则（如缓冲区占用率或带宽测量）的 ABR 算法，根据规则动态切换请求的视频码率；

本地缓存管理：将接收的分片临时存储至 download 文件夹，播放完成后自动清理。

扩展功能

智能码率自适应 (25 分)：使用机器学习或强化学习（推荐基础 Q-Learning 表格法，或时间序列预测、DQN 等）替代传统规则，根据网络波动动态输出码率决策，统计并对比其与基础算法的 QoE（用户体验质量）提升情况；

动态路径优化 (25 分)：集成简单的寻路算法优化传输路径，降低传输延迟并提升吞吐量；

滑动窗口流式处理 (15 分)：实现分片数据的滑动窗口重组机制，支持长视频流无缝播放；

端到端加密传输 (10 分)：使用 AES 加密视频分片，确保传输安全性；

学生自定义开放性问题 (10-25 分)：自己提出一个与“计算机网络架构”或“通信效率”相关的扩展问题，并给出代码实现方案。

技术栈要求：

编程语言：C/Python（推荐 C 语言，与实验框架兼容）；

通信协议：TCP（保证分片顺序与完整性），UDP 可选用于状态监控；

工具库：FFmpeg（视频分片处理）、Wireshark（网络抓包分析）；

数据存储：本地文件系统（分片存储）、SQLite（传输日志记录）。

分工建议：

- 服务端开发（分片生成、并发传输与日志管理） 1-2 人
- 客户端开发（分片请求、播放器集成与缓存） 1-2 人
- 算法设计（ABR 算法设计与实现、动态路径优化算法设计与实现） 1 人
- 测试与调优（网络波动模拟与性能分析） 1 人

实验提交内容：

1. 代码

服务端与客户端完整源码；

分片处理与 ABR 算法实现代码。

2. 文档

设计文档：系统架构图、分片协议格式、ABR 算法流程图；

测试报告：不同网络带宽下的卡顿率与 PSNR 对比分析；

用户手册：环境配置步骤与播放器调试说明。

3. 演示

实时展示分片传输与播放过程（至少 3 个客户端同时运行）；

模拟网络带宽骤降场景，展示码率自适应切换效果。

题目九：DDoS 攻击对抗与基于深度学习的智能防御系统设计

安全警告：本实验中所有的 DDoS 攻击模拟，仅限在本地局域网（127.0.0.1/私有网段）或 Mininet 虚拟网络拓扑中进行测试！严禁将目标 IP 设置为任何真实的公网服务器！

实验目标：

1. 理解 DDoS 攻击原理及常见攻击类型（如 SYN Flood、HTTP Flood）；
2. 掌握基于流量特征分析的攻击检测方法；
3. 实现基础防御策略（限速、黑名单、流量清洗）；
4. 探索 AI 驱动的网络攻击识别与自动化响应。

实验要求：

基础功能（必做）

1、攻击模拟模块：

使用工具（如 LOIC、hping3）模拟以下攻击类型：

SYN Flood：伪造随机源 IP 发送大量半连接请求；

HTTP Flood：模拟合法用户发起高频 GET/POST 请求；

UDP 反射放大攻击：伪造源 IP 向 DNS/NTP 服务器发送请求。

记录流量特征：利用 Wireshark 或 tcpdump 抓包，记录包速率（PPS）、协议分布、IP 熵值等统计信息。

2、防御系统模块：

基于 Linux 系统实现以下基础防御功能：

流量限速：通过 iptables 限制单个 IP 连接速率（如每秒 50 个 SYN 包）；

IP 黑名单：自动封锁异常 IP（如 1 分钟内发起 1000 次请求）；

流量清洗：使用 nftables 过滤非法协议包（如非 HTTP 端口的 80 请求）。

扩展功能

智能攻击分类器（25 分）：利用采集的 PCAP 数据提取统计特征（如每秒包数、平均包大小、SYN/ACK 比例）。构建一个简单的全连接神经网络（MLP），训练模型准确区分“正常流量”与“攻击流量”；

无监督异常检测（25 分）：假设应对未知攻击，使用 K-Means 聚类或自编码器（AutoEncoder）对流量特征进行分析，验证模型能否自动将高频攻击流量与正常背景流量分离；

自动化防御机制（10 分）：编写 Python/Bash 脚本实时监控网络接口或系统日志，当基于基础统计（如单 IP 请求频率超标）或联动 Snort（IDS）检测到攻击告警时，脚本自动执行 iptables 命令封禁该源 IP，实现从检测到阻断的运维自动化闭环；

CDN 集成（15 分）：配置 Nginx 反向代理实现请求分发与源 IP 隐藏；

学生自定义开放性问题（10-25 分）：自己提出一个与“计算机网络架构”或“通信效率”相关的扩展问题，并给出代码实现方案。

技术栈要求：

攻击工具：LOIC、hping3、Scapy；

防御工具：iptables/nftables、Nginx、Snort；

分析工具：Wireshark、Elastic Stack（流量可视化）；

编程语言：Python（自动化脚本）、Bash（防火墙规则管理）；

AI 框架：PyTorch（推荐）或 Scikit-learn；

数据存储：MySQL/Redis（攻击日志记录）。

分工建议：

- 攻击开发（模拟攻击并生成流量样本） 1-2 人
- 防御系统（配置防火墙规则与自动封禁脚本） 1-2 人
- 模型开发（流量特征提取与模型训练） 1 人
- 测试与文档（攻防对抗测试、撰写报告） 1 人

实验提交内容：

1. 代码

攻击脚本：包含模拟 SYN Flood 等攻击的 Shell 或 Python 脚本；

防御配置：包含 iptables/nftables 及 Nginx 规则配置文件；

算法与自动化源码（若选做）：特征提取代码、模型定义代码、预训练权重文件(.pth 或 .pkl)，以及实时监控流量并触发防御响应的 Python 脚本。

2. 文档

设计文档：系统架构图、DDoS 攻击流量统计特征表（需说明特征物理含义）；

测试报告：展示开启防御前后攻击成功率变化、CPU/内存负载的变化截图；

操作手册：环境配置步骤、脚本运行顺序及依赖库安装指南。

3. 演示

实时展示攻击流量生成与防御系统拦截效果（Wireshark 抓包对比）；

模拟混合攻击(SYN+HTTP Flood)并展示 AI 模型实时推理与自动封禁的联动效果(若选做)。

题目十：基于经典协议的邮件客户端开发与机器学习反垃圾机制

实验目标：

1. 掌握 SMTP 协议发送邮件与 POP3/IMAP 协议接收邮件的核心流程；
2. 实现 MIME 协议对邮件正文与附件的编码/解码；
3. 集成 SSL/TLS 加密机制保障邮件传输安全性；
4. 探索文本分类算法（如朴素贝叶斯）在垃圾邮件过滤中的应用。

实验要求：

基础功能（必做）

1、客户端功能：

邮件发送：通过 SMTP 协议发送纯文本/HTML 邮件，支持添加附件（如图片、文档）；

邮件接收：通过 POP3 协议获取邮箱邮件列表并下载内容；

用户认证：支持明文密码与 SSL 加密登录（如 LOGIN 与 AUTH PLAIN 命令）；

本地存储：将接收的邮件保存为 .eml 格式文件。

2、服务端功能：

模拟邮件服务器：使用 Python 的 smtpd 或 aiosmtpd 模块实现基础 SMTP 服务；

并发支持：支持多用户并发连接（至少 10 个客户端同时操作）；

邮件存储：使用 SQLite 数据库保存邮件元数据（发件人、收件人、时间戳）。

扩展功能

智能垃圾邮件过滤 (25 分): 实现基于文本分类模型的垃圾邮件过滤系统。推荐实现流程: 下载公开数据集 (如 Enron-Spam Dataset), 使用 Scikit-learn 库提取邮件文本的 TF-IDF 特征, 训练一个朴素贝叶斯 (Naive Bayes) 或 SVM 分类器。在客户端接收邮件时实时调用模型, 将识别为 Spam 的邮件自动移入“垃圾箱”;

邮件撤回功能 (10 分): 通过唯一邮件 ID 实现服务器端邮件撤回机制;

端到端加密 (PGP) (15 分): 使用 PGP/GPG 对邮件正文进行加密与数字签名验证;

Web 邮件管理界面 (25 分): 使用 Flask/Django 开发浏览器端的邮件收发界面, 替代传统的命令行/桌面客户端;

学生自定义开放性问题 (10-25 分): 自己提出一个与“计算机网络架构”或“通信效率”相关的扩展问题, 并给出代码实现方案。

技术栈要求:

协议标准: SMTP (RFC 5321)、POP3 (RFC 1939)、MIME (RFC 2045);

编程语言: Python (推荐使用 smtplib, poplib, email 库);

机器学习: Scikit-learn;

安全机制: SSL/TLS、PGP (GnuPG 库)。

分工建议:

- 基础协议开发 (SMTP 发送、POP3 接收、MIME 附件处理) 1-2 人
- 服务端开发 (模拟服务器构建、数据库存储) 1 人
- 智能扩展 (训练垃圾邮件模型) 1 人
- 测试与文档 (协议抓包分析与报告撰写) 1 人

实验提交内容:

1. 代码

客户端完整源码 (含 Socket 通信逻辑);

服务端脚本 (含并发处理与数据库操作);

扩展功能代码 (若选做), 如垃圾邮件分类器的训练脚本 train_spam.py 及保存的模型文件。

2. 文档

设计文档: 协议交互时序图、MIME 结构解析流程、AI 模型选型说明 (若选做);

测试报告: 并发压力测试结果 (如 50 用户同时发送), 垃圾邮件检测的准确率 (Accuracy) 与混淆矩阵分析图 (若选做);

用户手册: 客户端配置说明 (服务器 IP、端口、SSL 证书导入)。

3. 演示

发送带附件的邮件, 接收端完整解析并保存;

模拟中间人攻击, 展示 SSL 加密防护效果;

垃圾邮件分类演示 (若选做): 现场发送一封包含典型垃圾邮件特征 (如 "Win Lottery Now!") 的文本, 客户端接收后, AI 模型自动将其标记为 [SPAM] 并变色显示。